ReconcileUnderPolicy: A Repeatable Multi-Operator Benchmark for Policy-Constrained Kubernetes Recovery

Mandar Joshi
mandarjoshi1045@gmail.com

April 2026

Abstract

Kubernetes operators automate deployment, scaling, and recovery for complex stateful applications, but lightweight comparative benchmarks for recovery behavior are still uncommon. This paper presents *ReconcileUnderPolicy*, a standalone benchmark artifact for policy-constrained recovery across 2 stateful operators, 2 execution environments, and four recovery scenarios. The current study evaluates the RabbitMQ Cluster Operator and CloudNativePG on both single-node and multi-node `kind` clusters, running 3 repeated trials per study for 48 scenario executions in total. All 48 scenario executions converged successfully. Median completion times reveal substantial operator-specific differences: RabbitMQ baseline creation required 174.93 s and 189.58 s on single-node and multi-node clusters, while CloudNativePG required 38.04 s and 35.93 s. Delete-and-recreate recovery showed a similar gap, with RabbitMQ at 59.50 s and 72.81 s versus CloudNativePG at 32.17 s and 33.58 s. The benchmark also exposes warmup-sensitive variance: CloudNativePG baseline means were substantially higher than medians because the first trial in each environment was much slower than subsequent runs. The result is a reproducible, artifact-backed comparative benchmark that captures both successful recovery coverage and scenario-level timing behavior across operators and environments.

1 Introduction

Kubernetes operators encode operational logic into custom resources and controllers that provision, scale, update, and recover applications through reconciliation loops [4]. For stateful middleware, that control logic determines whether replicas recover correctly after restarts, whether policy constraints block progress cleanly, and how quickly the system converges back to a healthy state.

Recent work has shown that operator bugs are common and that automated testing frameworks can expose subtle controller failures [2, 6, 3, 7]. At the same time, there is still a practical gap between those research insights and a lightweight artifact that can be cloned, executed locally, and used to compare recovery behavior across multiple real operators. Many studies either emphasize broad bug finding or require infrastructure and orchestration that are heavier than what a small empirical benchmark needs.

This paper addresses that gap with *ReconcileUnderPolicy*, a standalone benchmark artifact for comparative operator-recovery evaluation. The current study evaluates two real stateful operators, RabbitMQ Cluster Operator and CloudNativePG [5, 1], across both single-node and multi-node `kind` environments. Each study executes the same four scenarios: baseline cluster creation, operator restart during scale-up, ResourceQuota-blocked scale-up, and delete-and-recreate recovery.

The contribution is both empirical and methodological:

- the repository executes a shared scenario suite across multiple real operators and environments;
- it captures repeated-trial timing data rather than isolated one-off runs;
- it packages machine-readable results and LaTeX-ready tables directly from the benchmark outputs;
- it exposes where recovery behavior is stable, where it is operator-specific, and where warmup effects distort means.

The resulting paper is therefore not a claim about all Kubernetes operators. Instead, it is a reproducible comparative benchmark study that demonstrates how policy-constrained recovery can be evaluated with a small but meaningful matrix of operators and environments.

2 Benchmark Scope

2.1 Research Questions

The benchmark focuses on three questions:

1. Can a lightweight local artifact repeatedly execute non-trivial recovery scenarios across multiple real stateful operators?
2. How do recovery times differ across operators and environments when the scenario suite is held constant?
3. What kinds of run-to-run variability become visible once repeated trials are collected rather than single-run anecdotes?

2.2 Study Matrix

The present evaluation spans 4 studies:

- operators: RabbitMQ Cluster Operator and CloudNativePG;
- environments: single-node `kind` and multi-node `kind`;
- trials: 3 full trials per operator/environment pair;
- scenarios: four recovery scenarios per trial.

This produces 12 trial runs and 48 total scenario executions.

2.3 Scenario Suite

Each trial executes four scenarios in sequence:

1. **baseline_create**: create a one-replica managed cluster and measure time to first healthy reconciliation;
2. **restart_during_scale**: scale from one to two replicas while deleting the operator pod mid-reconcile;
3. **quota_blocked_scale**: apply a temporary ResourceQuota that blocks creation of the next pod during scale-up, then remove it and measure recovery;
4. **delete_and_recreate**: delete the managed cluster, wait for cleanup, recreate it, and measure time back to a healthy one-replica state.

3 Artifact Design

3.1 Repository Structure

The repository is organized into three main areas:

- `vendor/cluster-operator-main`: vendored RabbitMQ Cluster Operator source used to render local install manifests;
- `vendor/cloudnative-pg`: vendored CloudNativePG release assets;
- `benchmark/reconcileunderpolicy/`: benchmark scripts, environment definitions, study outputs, and reproduction steps;
- `paper/`: LaTeX source, generated tables, and bibliography.

3.2 Execution Environment

The benchmark runs on two dedicated `kind` profiles: a single-node cluster and a three-node cluster with one control-plane node and two workers. RabbitMQ and CloudNativePG are both exercised with the same per-instance resource envelope: 200m CPU and 512Mi memory requests, plus 500m CPU and 1Gi memory limits. This keeps the scenarios comparable while remaining feasible on commodity local hardware.

RabbitMQ installation is rendered locally from the vendored source tree. CloudNativePG installation uses the vendored `v1.29.0` release manifest applied with server-side `kubectl`. Managed workload images are `rabbitmq:4.1.3-management` for RabbitMQ. CloudNativePG uses `ghcr.io/cloudnative-pg/postgresql:17`.

3.3 Outputs

Each study run creates a timestamped directory containing:

- per-trial summaries,
- per-scenario snapshots of pods, PVCs, services, and events,
- aggregate JSON, CSV, and Markdown summaries,
- generated LaTeX include files for the manuscript tables.

4 Methodology

Each study begins from a clean workload state, ensures the operator deployment is healthy, and then runs the four scenarios in order. Success is defined as convergence to the expected healthy replica count for the scenario under test.

For each scenario, the benchmark records:

- end-to-end completion time,
- final ready replica count,
- scenario-specific metadata such as the deleted operator pod name or the ready-replica count observed while the quota block was active.

Aggregated statistics are computed per scenario for each operator/environment pair:

- successful trials out of total trials,
- mean completion time,

- median completion time,
- standard deviation,
- minimum and maximum completion times.

Both mean and median are reported because several scenarios exhibit clear first-trial skew. Means capture the overall cost actually observed in the study, while medians better reflect steady repeated behavior when a single cold start dominates one run.

5 Results

Table 1 summarizes the per-study median completion times for the four scenarios. Table 2 then reports the full aggregate statistics for each operator/environment pair.

Table 1: Median completion time in seconds by operator and environment

Operator	Env.	Baseline	Restart	Quota	Delete
RabbitMQ	Single-node	174.93	77.61	79.54	59.50
RabbitMQ	Multi-node	189.58	88.74	92.97	72.81
CloudNativePG	Single-node	38.04	27.72	54.27	32.17
CloudNativePG	Multi-node	35.93	31.50	54.13	33.58

Table 2: Aggregate statistics across 3 trials per study

Operator	Env.	Scenario	Success	Mean (s)	Stddev (s)	Range (s)
RabbitMQ	Single-node	Baseline create	3/3	175.00	0.20	174.80–175.27
RabbitMQ	Single-node	Restart during scale	3/3	77.16	1.48	75.17–78.71
RabbitMQ	Single-node	Quota-blocked scale	3/3	80.23	1.09	79.37–81.77
RabbitMQ	Single-node	Delete and recreate	3/3	59.49	0.07	59.41–59.57
RabbitMQ	Multi-node	Baseline create	3/3	192.33	3.96	189.49–197.93
RabbitMQ	Multi-node	Restart during scale	3/3	149.33	85.80	88.58–270.66
RabbitMQ	Multi-node	Quota-blocked scale	3/3	93.53	0.87	92.85–94.76
RabbitMQ	Multi-node	Delete and recreate	3/3	72.82	0.13	72.67–72.98
CloudNativePG	Single-node	Baseline create	3/3	103.39	94.31	35.37–236.76
CloudNativePG	Single-node	Restart during scale	3/3	28.78	1.64	27.53–31.09
CloudNativePG	Single-node	Quota-blocked scale	3/3	52.93	2.07	50.00–54.51
CloudNativePG	Single-node	Delete and recreate	3/3	33.05	1.25	32.16–34.81
CloudNativePG	Multi-node	Baseline create	3/3	114.38	111.13	35.68–271.54
CloudNativePG	Multi-node	Restart during scale	3/3	102.06	101.44	29.16–245.51
CloudNativePG	Multi-node	Quota-blocked scale	3/3	54.71	1.02	53.85–56.15
CloudNativePG	Multi-node	Delete and recreate	3/3	32.94	0.92	31.63–33.60

Four observations stand out.

All scenario executions converged successfully. Every one of the 48 scenario executions succeeded, including operator restart during scale-up, temporary policy blocking, and full delete-and-recreate recovery. The artifact therefore exercises disruptive control-plane events without requiring manual intervention or operator-specific special casing in the paper workflow.

CloudNativePG recovered faster than RabbitMQ in baseline and delete-and-recreate scenarios. On single-node `kind`, median baseline creation was 38.04 s for CloudNativePG versus 174.93 s for RabbitMQ. On multi-node `kind`, the same comparison was 35.93 s versus 189.58 s. Delete-and-recreate showed a similar pattern: 32.17 s and 33.58 s for CloudNativePG versus 59.50 s and 72.81 s for RabbitMQ. That gap suggests the benchmark is measuring genuine operator-specific behavior rather than only generic cluster overhead.

Environment effects were operator-dependent rather than uniform. RabbitMQ slowed down on the multi-node profile across all four scenarios, including restart-during-scale median

growth from 77.61 s to 88.74 s and delete-and-recreate growth from 59.50 s to 72.81 s. CloudNativePG was much less sensitive in median terms: baseline medians were 38.04 s and 35.93 s, while delete-and-recreate medians were 32.17 s and 33.58 s. The environment tax is therefore not a fixed property of the benchmark harness; it depends on the operator and scenario.

Means exposed warmup-sensitive outliers that medians would hide. CloudNativePG baseline creation had means of 103.39 s and 114.38 s but medians of only 38.04 s and 35.93 s, driven by a much slower first trial in each environment. RabbitMQ multi-node restart during scale showed a similar pattern, with an 88.74 s median but a 149.33 s mean due to one 270.66 s trial. Reporting both measures is therefore important: the means preserve the full observed cost, while the medians expose the more typical steady-state recovery time.

Quota-blocked recovery was consistently observable and stable. All four studies successfully stalled progress while the ResourceQuota was in place and then recovered once the quota was removed. Standard deviations stayed low for this scenario across the matrix, ranging from 0.87 s to 2.07 s. This makes quota-blocked scale-up a useful cross-operator stressor for future work because it creates a real controller-visible perturbation without modifying the operators themselves.

6 Discussion

6.1 What the Study Establishes

This work does not claim to characterize the entire operator ecosystem. It does establish that a lightweight repository can support a non-trivial comparative matrix over real operators and environments while still remaining executable on local infrastructure. The benchmark now moves beyond a single-operator pilot: it produces comparative tables, captures repeated-trial variance, and exposes where operator recovery behavior diverges under the same perturbation suite.

6.2 Threats to Validity

The claims of the paper are intentionally scoped, and the main validity limits concern generalization.

- **External validity.** The study covers only two operators, so the measured recovery behaviors should not be generalized to all operator ecosystems without broader sampling.
- **Environment validity.** Both environments are local `kind` clusters rather than production deployments, so absolute timings are environment-specific even though the workflows are realistic.
- **Policy validity.** The current perturbation suite centers on ResourceQuota-based interference and controller restarts. Other policy mechanisms such as admission rejection or RBAC denial are not yet included.
- **Construct validity.** The benchmark measures convergence and recovery timing, not deeper workload semantics beyond operator health and replica readiness.
- **Warmup effects.** Several scenarios showed first-trial skew, which means future studies should control for image pull, initialization, or storage warmup when comparing absolute means.

These constraints bound the conclusions, but they do not undermine the central contribution: a reproducible comparative benchmark for policy-constrained operator recovery with real execution traces and repeated measurements.

6.3 Next Steps

The most promising extensions are:

1. add more operators while preserving the same scenario suite;
2. incorporate additional policy mechanisms such as admission denial or RBAC restriction;
3. extend the environments beyond local `kind` clusters;
4. correlate timing differences with richer controller-log and event-trace analysis.

7 Related Work

Acto [2] introduced automated end-to-end testing for operation correctness in cloud system management. Sieve [6] extended this line of work with perturbation-based testing for Kubernetes controllers. The NSDI 2026 reliability study [3] and the ISSTA 2024 operator bug study [7] provide broader motivation by showing that operator failures are both common and consequential.

The present work is narrower than those systems papers. Its contribution lies in packaging a runnable comparative benchmark artifact around real stateful operators, exposing policy-constrained recovery behavior across repeated trials, and turning those outputs directly into paper-ready comparative tables.

8 Conclusion

ReconcileUnderPolicy now provides a repeatable comparative benchmark for policy-constrained recovery in Kubernetes operators. Across 4 studies, 12 repeated trial runs, and 48 scenario executions, the artifact successfully evaluated RabbitMQ Cluster Operator and CloudNativePG on both single-node and multi-node `kind` profiles.

The resulting data show that recovery behavior is meaningfully operator-dependent: CloudNativePG converged substantially faster than RabbitMQ in baseline and delete-and-recreate scenarios, RabbitMQ exhibited a clearer multi-node penalty, and several scenarios displayed warmup-sensitive outliers that would be missed by single-run evaluation. The repository therefore supports a stronger publication claim than a pilot artifact alone: it offers a reproducible methodology, real comparative data, and a reusable baseline for larger studies of policy-constrained operator recovery.

References

[1] CloudNativePG. Cloudnativepg documentation. `https://cloudnative-pg.io/documentation/`. Accessed April 26, 2026.

[2] Jiawei Tyler Gu, Xudong Sun, Wenqing Zhang, Yuqing Jiang, Chen Wang, Mandana Vaziri, Owolabi Legunsen, and Tianyin Xu. Acto: Automatic end-to-end testing for operation correctness of cloud system management. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023.

[3] Jiawei Tyler Gu, Zhen Tang, Yiming Su, Bogdan Alexandru Stoica, Xudong Sun, William X. Zheng, Yue Zhang, Akond Rahman, Chen Wang, and Tianyin Xu. Who watches the watchers? on the reliability of softwarizing cloud application management. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, 2026.

[4] Kubernetes Documentation. Operator pattern. `https://kubernetes.io/docs/concepts/extend-kubernetes/operator/`. Accessed April 26, 2026.

[5] RabbitMQ. Rabbitmq cluster kubernetes operator quickstart. https://www.rabbitmq.com/kubernetes/operator/quickstart-operator. Accessed April 26, 2026.

[6] Xudong Sun, Wenqing Luo, Jiawei Tyler Gu, Aishwarya Ganesan, Ramnatthan Alagappan, Michael Gasch, Lalith Suresh, and Tianyin Xu. Sieve: Chaos testing for kubernetes controllers. USENIX ;login: Online, 2024. https://www.usenix.org/publications/loginonline/sieve-chaos-testing-kubernetes-controllers.

[7] Qingxin Xu, Yu Gao, and Jun Wei. An empirical study on kubernetes operator bugs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.